A Perceptual-Dynamic Gap Bridging Pipeline for Quadrupedal Visual Navigation using 3D Gaussian Splatting and RL-Induced Jitter

Minseok Lee[1], Seongyu Han[1], Heejae Moon[1], and Sung Soo Hwang[1*]

[1]School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

Abstract: Directly validating and deploying autonomous visual navigation systems on physical quadrupedal robots in real-world settings presents severe challenges, specifically the perception-level sim-to-real gap (lack of visual photorealism in synthetic environments) and locomotion-induced camera shaking (ego-motion jitter). This vibration degrades RGB-D feature tracking and depth consistency. To resolve these challenges safely and efficiently, this paper proposes a practical pipeline that bridges these perceptual and dynamic gaps to enable virtual validation and direct parameter optimization. In the virtual sandbox stage, we reconstruct a high-fidelity 3D Gaussian Splatting (3DGS) digital twin of an actual campus corridor and utilize a reinforcement learning (RL) locomotion policy in simulation to actively emulate walking dynamics and generate representative camera bobbing noise. Under these synchronized visual and dynamic disturbances, the parameters of the RTAB-Map visual SLAM and Nav2 stacks are pre-tuned and optimized. In the deployment stage, the RL policy is bypassed, and navigation velocity commands are routed directly to the physical robot's built-in Sport API via a custom ROS 2 bridge. Experimental results in the actual corridor demonstrate that parameters optimized under the simulated RL-induced jitter transfer directly to the physical platform, achieving stable visual mapping and autonomous obstacle avoidance without real-world parameter recalibration.

Keywords: Quadrupedal Locomotion, Visual SLAM, Sim-to-Real Pipeline, 3D Gaussian Splatting, Ego-motion Jitter, Virtual Sandbox.

1. INTRODUCTION

Autonomous quadrupedal robots offer superior mobility in traversing challenging terrains compared to conventional wheeled systems. Achieving indoor autonomy requires the integration of legged locomotion and simultaneous localization and mapping (SLAM). Indoor navigation has conventionally relied on Light Detection and Ranging (LiDAR) sensors. However, LiDARs are costly, heavy, and power-intensive, making visual navigation using a single lightweight RGB-D camera a desirable and cost-effective alternative.

However, visual navigation on legged platforms introduces two major sim-to-real challenges. First is the *perceptual sim-to-real gap*: standard simulators lack photorealistic visual textures and depth fidelity, causing feature-based visual SLAM to fail in the real world due to matching errors. Second is *locomotion-induced ego-motion noise* (camera bobbing): legged locomotion generates periodic oscillations that cause motion blur and temporal depth discrepancies (phantom obstacles). Furthermore, tuning parameters directly on physical hardware is constrained by battery capacity and the risk of collision-induced damage to expensive platforms.

To address these challenges, we propose a practical, bifurcated pipeline for quadrupedal visual navigation using a single front-facing RGB-D camera (Intel RealSense D435i). Our framework decouples virtual-space parameter verification from physical-space deployment. By using DSLR photos for offline 3DGS reconstruction and matching the virtual camera's parameters with the physical sensor, we minimize sensor discrepancy. In the simulation phase, we reconstruct a photorealistic digital twin of an indoor corridor. Because the robot's low-level controller cannot be integrated directly into the physics simulator, we train a model-free RL locomotion policy in Isaac Lab to emulate physical gait characteristics, generating representative camera ego-motion noise. Under this sandbox, we optimize the parameters of the RTAB-Map and Nav2 frameworks. Upon physical deployment, the computationally heavy RL policy is bypassed, and navigation commands are routed directly to the manufacturer's built-in Sport API via a ROS 2 bridge, saving edge computing resources for visual SLAM and navigation.

The contributions of this work are summarized as follows:

1. We present a system integration pipeline that bridges the perceptual sim-to-real gap by incorporating DSLR-based 3DGS and *fVDB* reconstruction into a physics simulator, establishing a photorealistic visual validation sandbox where virtual camera parameters are matched with the physical RGB-D camera.
2. We bridge the dynamic sim-to-real gap by training an RL locomotion policy to serve as an active camera jitter generator, recreating quadruped-specific walking oscillations in the virtual sandbox.
3. We demonstrate the feasibility of the pipeline by showing that navigation and mapping parameters optimized under simulated visual and dynamic disturbances transfer directly to a physical Unitree Go2 platform without real-world recalibration.

2. RELATED WORK

2.1 Reinforcement Learning for Quadrupedal Locomotion

Reinforcement learning (RL) has advanced control paradigms for legged systems [1]. While classical methods like Model Predictive Control (MPC) rely heavily on accurate dynamics, model-free RL enables robots to acquire robust policies. Highly parallelized simulators, such as NVIDIA Isaac Sim and Isaac Lab, facilitate training deep RL policies across thousands of concurrent environments, enabling successful sim-to-real transfer on systems like ANYmal and Unitree Go1/Go2. However, most locomotion research focuses primarily on gait stability, leaving high-level visual navigation integration and sensor noise characteristics unaddressed.

2.2 Visual SLAM and Navigation Integration

Mobile robot navigation has classically relied on LiDAR sensors. However, LiDARs are heavy and power-intensive, making them less suitable for payload-constrained quadruped robots. Visual SLAM (VSLAM) utilizing RGB-D cameras offers a lightweight alternative. Among VSLAM frameworks, Real-Time Appearance-Based Mapping (RTAB-Map) [2] is widely adopted for its appearance-based loop closure detection and pose-graph optimization. Simultaneously, the ROS 2 Navigation (Nav2) framework [3] serves as the industry standard for path planning and obstacle avoidance.

Although works like VR-Robo [5] leverage 3DGS [4] to build digital twins, they often rely on heterogeneous sensors (e.g., using an iPad for modeling and an action camera for deployment). This introduces device-level sensor discrepancies, such as mismatched lens distortions and field-of-view (FOV) boundaries. In contrast, our pipeline uses a DSLR camera for offline 3DGS reconstruction and configures the virtual camera to match the physical D435i RGB-D camera. This parameter synchronization minimizes visual domain discrepancies.

3. SYSTEM AND LOCOMOTION CONTROL

3.1 System Architecture

Our framework integrates RTAB-Map, Nav2, and a bifurcated locomotion control architecture. In simulation, a DRL policy—trained via Proximal Policy Optimization (PPO) in Isaac Lab (v2.3.2) on Isaac Sim (v5.1.0)—generates representative quadrupedal gait dynamics and camera ego-motion (bobbing) noise. High-level velocity commands ($\mathbf{v_{cmd}} = [\mathbf{v_x}, \mathbf{v_y}, \omega_\mathbf{z}]^\mathbf{T}$) from the Nav2 local planner are mapped directly to the policy's action space. During real-world deployment, the heavy DRL policy is bypassed to conserve resources, and velocity commands are routed directly to the Unitree Sport API via a ROS 2 bridge. This architecture optimizes the computational budget of the Jetson Orin NX. The system architecture is shown in Fig. 1.

3.2 Locomotion Learning in Isaac Sim

To simulate camera bobbing noise, we train an RL policy in NVIDIA Isaac Lab (v2.3.2) on the Isaac Sim (v5.1.0) physics engine. The policy functions as a dynamic perturbation generator, producing walking-induced oscillations. We construct a training environment with mixed terrains (uneven ground, ramps, step obstacles) using the Procedural Terrain Generator and scale terrain difficulty via curriculum learning. To enhance policy robustness, we apply the following domain randomizations:

- Torso mass randomization: $\Delta m \in [-1.0, +3.0]$ kg.
- Ground friction coefficient: $\mu \in [0.6, 0.8]$.
- External disturbances: impulse force perturbations applied to the robot's center of mass (CoM).

The physics step is 200 Hz, and the policy executes at 50 Hz. Training uses the PPO implementation in the PyTorch-based *skrl* library. Both actor and critic networks are Multilayer Perceptrons (MLPs) with hidden layers of $[512, 256, 128]$ and ELU activation functions. The learning rate is 1.0×10^{-3}, regulated by a KL-divergence scheduler. The discount factor is $\gamma = 0.99$, and the GAE parameter is $\lambda = 0.95$.

The reward function tracks velocity references while maintaining legged stability. The total reward R is:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

- $R_{lin} = \exp(-\|\mathbf{v}^*_{\mathbf{xy}} - \mathbf{v}_{\mathbf{xy}}\|^2_2/\sigma^2_{\mathbf{lin}})$ is the linear velocity tracking reward ($\mathbf{v}^*_{\mathbf{xy}}$ and $\mathbf{v}_{\mathbf{xy}}$ are target and actual velocities, $w_{lin} = 1.5, \sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega^*_z - \omega_z)^2/\sigma^2_{ang})$ is the angular velocity tracking reward (ω^*_z and ω_z are target and actual yaw rates, $w_{ang} = 0.75, \sigma_{ang} = 0.25$).
- R_{air} is the foot clearance air-time reward ($w_{air} = 0.25$).
- P_i represents penalty terms for base vertical velocity, roll/pitch oscillations, joint limits, excessive torque, joint acceleration, and unintended contact ($w_{p,i}$ are penalty weights).

3.3 3DGS Environment Construction & Agent Deployment

To construct the virtual sandbox, we capture high-resolution corridor images using a DSLR camera, avoiding mobile device compression artifacts. These images serve as input to COLMAP for Structure-from-Motion (SfM) to resolve camera poses. Then, a 3DGS [4] model is optimized and exported as a USDZ asset for visual rendering, alongside a PLY mesh for coarse geometry. To bridge the gap between photorealistic visualization and physical collision, the scene is integrated with NVIDIA's sparse volumetric framework, *fVDB*, as shown in Fig. 2.

We import these assets into Isaac Sim (v5.1.0), where the NuRec engine utilizes the volumetric *fVDB* representation to compute collision boundaries. A ROS 2 bridge publishes virtual RGB-D sensor streams, visualizes the

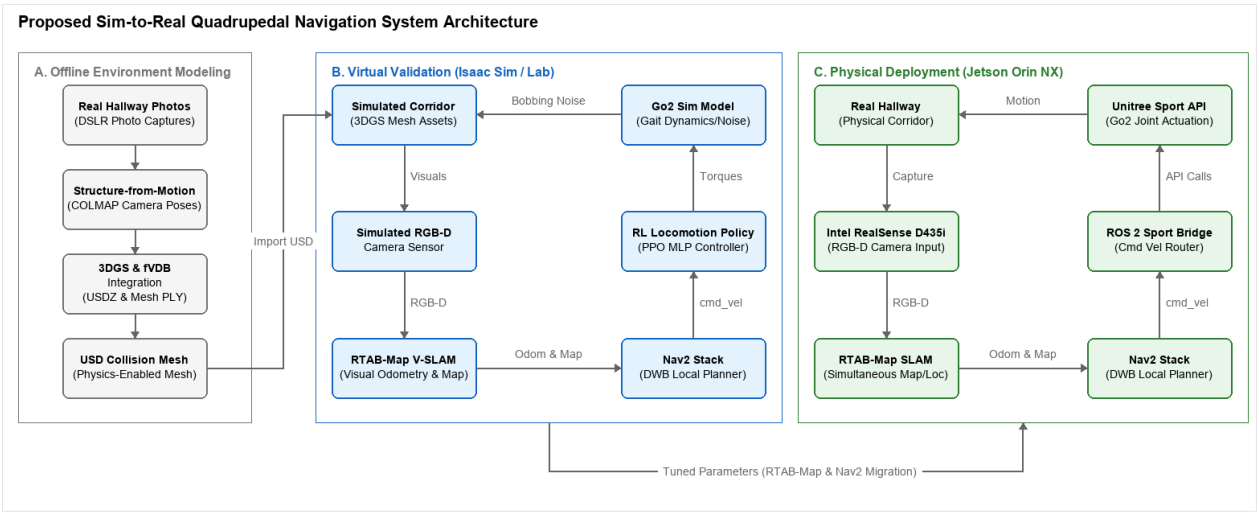

Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we build a high-fidelity 3DGS environment and train an RL locomotion policy to simulate camera bobbing noise for upper-level navigation tuning. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to optimize computational resources on the Jetson Orin NX.

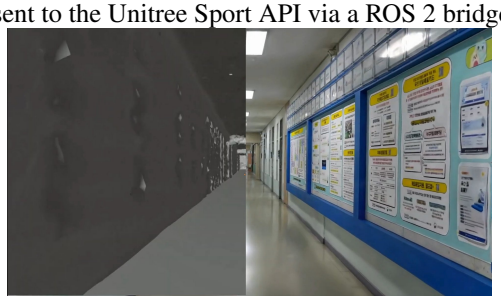

Fig. 2. Digital twin representation of the campus corridor. Left: Sparse volumetric collision mesh generated using fVDB. Right: High-fidelity photorealistic 3DGS render.

robot state in RViz, and routes velocity commands. The virtual camera's field-of-view and resolution are configured to match the physical D435i camera, minimizing the perceptual domain shift. The pre-trained DRL locomotion policy is then deployed directly on the Go2 model in simulation. Despite the narrow corridor boundaries, the policy maintains robust gait characteristics, validating the effectiveness of the domain-randomized training.

4. AUTONOMOUS NAVIGATION

4.1 Navigation System Configuration

The navigation system consists of an Intel RealSense D435i RGB-D sensor, the RTAB-Map framework [2], and the ROS 2 Nav2 stack. The sensor is mounted to the front torso of the Go2. In the virtual sandbox, the simulated camera replicates color and depth profiles. Using a single RGB-D sensor guarantees depth-texture consistency. Unlike monocular reconstruction frameworks (e.g., VR-Robo [5]) that face geometric holes in textureless regions due to photometric ambiguities, the D435i's active depth sensor allows us to project robust metric local costmaps.

Initially, RTAB-Map processes the RGB stream to extract visual keypoints (e.g., GFTT/ORB) for visual odometry (VO), loop closure, and pose-graph optimization. The depth stream is projected to construct a 3D octomap and a 2D occupancy grid. For low-latency obstacle avoidance, raw depth frames are converted into a pseudo-2D laser scan via the *depthimage_to_laserscan* ROS 2 package and published to the */scan* topic. The Nav2 global planner generates paths on the occupancy grid, while the DWB local planner uses the local costmap to compute steering commands $\mathbf{v_{cmd}}$, which are sent to the controller.

A key challenge is resolving the coordinate transform (TF) tree discrepancy between the physics simulator and ROS 2. Nav2 expects *map* $\rightarrow$ *odom* $\rightarrow$ *base*, whereas Isaac Sim tracks the robot relative to the simulator's *world* frame. To prevent conflicts, we implement a custom bridge resulting in a TF tree structured as *map* $\rightarrow$ *odom* $\rightarrow$ *world* $\rightarrow$ *base*. RTAB-Map publishes the *map* $\rightarrow$ *odom* transform, while Isaac Sim links the odometry to the world frame, and the robot state publisher completes the chain. This enables standard ROS 2 navigation algorithms to operate in simulation without modification.

4.2 Validation in 3DGS Environment

We validated the integrated navigation stack in the 3DGS-based virtual corridor (80 m long, 2.5 m wide) before physical deployment.

In simulation, the policy receives proprioceptive (joint states, angular velocity, gravity projection) and exteroceptive (a 1.6 m $\times$ 1.0 m height-scan grid) observations. Ray-casting queries the 3DGS collision mesh to feed local elevation profiles to the policy for adaptive foot placement. The velocity commands are supplied by the DWB

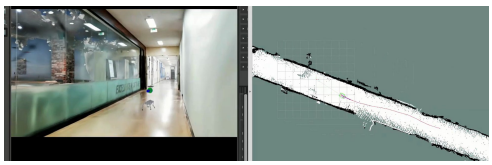

Fig. 3. Autonomous navigation execution inside the reconstructed virtual environment, showing the simulated quadruped robot in Isaac Sim (left) and path planning/costmap synchronization in RViz (right).

planner, which coordinates collision-free paths (Fig. 3).

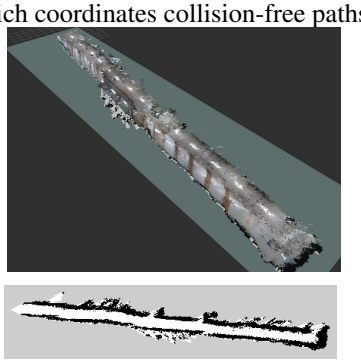

Fig. 4. Visual mapping results in the virtual sandbox. Top: Reconstructed 3D point cloud map from RTAB-Map. Bottom: Projected 2D occupancy grid map.

To evaluate obstacle avoidance, virtual box obstacles were placed along the corridor. The local planner dynamically recalculated trajectories, and the RL policy adjusted its walking pattern to negotiate detours (Fig. 3). The resulting point cloud and occupancy grid maps are shown in Fig. 4.

4.3 Physical Deployment and Results

Following virtual validation, the pipeline was deployed on a physical Unitree Go2 platform in Oseok Hall corridor (Fig. 5). The onboard hardware consists of an NVIDIA Jetson Orin NX (20W mode, 16GB RAM) running Ubuntu 20.04 and ROS 2 Foxy. To satisfy resource constraints, the computationally heavy DRL locomotion policy was bypassed. Instead, Nav2 velocity commands were routed directly to the robot's proprietary motor controller via the Unitree Sport API using a custom ROS 2 bridge. This reserves Jetson Orin NX resources for RTAB-Map visual SLAM and Nav2 planning. The D435i sensor was connected via USB 3.0.

Fig. 5. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

A practical challenge is the OS and ROS 2 version mismatch: simulation used Ubuntu 24.04/ROS 2 Jazzy, while the robot's onboard computer is restricted to Ubuntu 20.04/ROS 2 Foxy. To address this, the visual navigation stack was migrated to ROS 2 Foxy to run natively on the Jetson Orin NX. Despite the version differences, the system architecture remained identical, allowing navigation parameters optimized under simulated RL ego-motion noise (costmap inflation radius, voxel grid resolution, loop-closure thresholds) to be transferred directly without manual recalibration.

As shown in Fig. 5, RTAB-Map successfully executed loop closure detection, compensating for camera oscillations. Nav2 maintained stable global paths at a target velocity of 0.4 m/s. The results verify that parameters optimized under simulated RL ego-motion noise transfer directly to the physical platform, validating the utility of our sandbox.

5. CONCLUSION

This paper presented a practical, dual-stage visual navigation framework for quadrupedal robots. By leveraging a high-fidelity 3D Gaussian Splatting digital twin and simulating quadrupedal locomotion noise via an RL locomotion policy, we established a safe, photorealistic virtual validation sandbox for visual SLAM and path planning. For physical deployment, we bypassed the RL policy and routed commands directly to the robot's built-in Sport API via a custom ROS 2 bridge, optimizing edge computation on the Jetson Orin NX. Real-world experiments on the Unitree Go2 demonstrated that the parameters pre-tuned under simulated bobbing noise transferred successfully without manual in-situ recalibration, achieving robust visual tracking and obstacle avoidance.

Future work will focus on incorporating dynamic obstacle avoidance strategies and optimizing the processing pipeline to support higher-resolution visual feedback on the edge computer.

USE OF AI TOOLS

During the preparation of this work, the authors used Gemini (developed by Google) for grammar editing and style improvement of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the final text of the paper.

ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[2] M. Labbé and F. Pomerleau, "Online global loop closure detection for large-scale multi-session graph-based SLAM," *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

[5] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, "VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion," *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 7875–7882, August 2025.